



CortexAI `proxyWithHeader` Utility Documentation

This document explains the complete why, what, how, and the cross-file parameter journey of the `proxyWithHeader` utility function located at `gateway/utils/proxywithheader.js`.

1. What is `proxyWithHeader` ? (For Beginners)

In CortexAI's microservices architecture, the Gateway (port `8000`) forwards requests to downstream services like Chat (port `8002`) and Agent (port `8003`). But there is a critical problem:

Problem: When the Gateway proxies a request to the Chat or Agent service, those downstream services have no idea *who* the user is. The session cookie was already validated and consumed by the Gateway — the downstream service never sees it.

`proxyWithHeader` solves this by acting as:

1. **A Smart Reverse Proxy:** It forwards the entire HTTP request to a target microservice URL, just like a normal proxy.
2. **An Identity Bridge:** Before forwarding, it *decorates* the outgoing request by injecting a custom HTTP header (`x-user-id`) containing the authenticated user's database ID.
3. **A Safety Guard:** It throws an error at server startup if the target service URL is missing from `.env`, preventing silent proxy failures at runtime.

Why not just use a normal `express-http-proxy` ?

The plain `proxy()` from `express-http-proxy` (used for the `/auth` route) forwards requests as-is — no modifications. But `/chat` and `/agent` routes are **protected** — they need to know which user is making the request. `proxyWithHeader` wraps the plain proxy and injects user identity into the forwarded request headers.

2. The Complete Source Code (Line-by-Line Breakdown)

File: `gateway/utils/proxywithheader.js`

```
import proxy from "express-http-proxy" // Line 1

export const proxyWithHeader = (serviceUrl) => { // Line 3
  if (!serviceUrl) { // Line 4
    throw new Error("proxyWithHeader: serviceUrl is ...") // Line 5
  } // Line 6

  return proxy(serviceUrl, { // Line 8
    proxyReqOptDecorator: (proxyReqOpts, srcReq) => { // Line 9
      if (srcReq.user) { // Line 10
        proxyReqOpts.headers['x-user-id'] = srcReq.user.userid // Line 11
      } // Line 12
      return proxyReqOpts // Line 13
    }, // Line 14
  }) // Line 15
} // Line 16
```

3. Variables & Parameters Dictionary

Function Signature: `proxyWithHeader(serviceUrl)`

- `serviceUrl` (String — the function's input parameter):
 - **What it is:** The full URL of the target microservice to proxy requests to.
 - **Where it comes from:** Passed in from `gateway/index.js` via environment variables.
 - **Possible values:**
 - `process.env.CHAT_SERVICE` → `"http://localhost:8002"` (from `gateway/.env`)
 - `process.env.AGENT_SERVICE` → `"http://localhost:8003"` (from

`gateway/.env`)

- **What happens if empty:** The function throws a descriptive `Error` at server startup, preventing the Gateway from starting with a broken proxy configuration.

Callback: `proxyReqOptDecorator(proxyReqOpts, srcReq)`

This is a hook provided by the `express-http-proxy` library. It fires **after** the proxy decides to forward the request but **before** the request is actually sent to the target service.

- `proxyReqOpts` (Object — provided by `express-http-proxy`):
 - **What it is:** The outgoing HTTP request options object that the proxy is about to send to the target service.
 - **Key property used:** `proxyReqOpts.headers` — the headers object of the outgoing proxied request.
 - **What we do with it:** We add a new custom header `x-user-id` to this object before it is sent.
- `srcReq` (Object — the original Express `req` object):
 - **What it is:** The original incoming HTTP request from the browser, as received by the Gateway. This is the same `req` object that passed through the `protect` middleware.
 - **Key property used:** `srcReq.user` — this is the user session data that was attached by `protect` middleware.

The Injected Header: `x-user-id`

- `srcReq.user.userId` (String — MongoDB ObjectId):
 - **What it is:** The MongoDB `_id` of the authenticated user from the `users` collection in the `auth` database.
 - **Header name:** `x-user-id`
 - **Who reads it downstream:**
 - Chat Service's `chat.controller.js` reads `req.headers["x-user-id"]`
 - Agent Service's `agent.controller.js` receives it (available but currently unused in code)
-

4. The Complete Cross-File Parameter Journey

This is the exact origin story of every value that flows through `proxyWithHeader`, traced across **6 files** in the project:

Journey of `serviceUrl`

```
[Step 1] gateway/.env
| CHAT_SERVICE=http://localhost:8002
| AGENT_SERVICE=http://localhost:8003
▼
[Step 2] gateway/index.js (Line 4: dotenv.config() loads .env into process.env)
| process.env.CHAT_SERVICE → "http://localhost:8002"
| process.env.AGENT_SERVICE → "http://localhost:8003"
▼
[Step 3] gateway/index.js (Lines 23–24: passed as argument to proxyWithHeader)
| proxyWithHeader(process.env.CHAT_SERVICE) // serviceUrl = "http://localhost:8002"
| proxyWithHeader(process.env.AGENT_SERVICE) // serviceUrl = "http://localhost:8003"
▼
[Step 4] gateway/utils/proxywithheader.js (Line 3: received as parameter)
| proxyWithHeader = (serviceUrl) => { ... }
| serviceUrl is passed to proxy(serviceUrl, ...) on Line 8
▼
[Step 5] express-http-proxy library
| proxy() uses serviceUrl to forward the HTTP request to that URL.
```

Journey of `srcReq.user.userid`

This is the most important data flow — tracing where the user's database ID comes from:

[Step 1] services/auth/controller/auth.controller.js (Lines 12-24)

- | Firebase verifies the Google ID token.
- | MongoDB finds or creates the User document.
- | user._id = MongoDB-generated ObjectId (e.g. "6832f5a1...")



[Step 2] services/auth/controller/auth.controller.js (Lines 26-32)

- | Stores session data in Redis:
- | redis.set("session-<uuid>", JSON.stringify({
- | userid: user._id, ← This is the key field
- | name: user.name,
- | email: user.email,
- | avatar: user.avatar
- | }), "EX", 604800)



[Step 3] Browser Cookie

- | Auth controller sets cookie: res.cookie("session", sessionid, {...})
- | Browser stores cookie and sends it with every subsequent request.



[Step 4] gateway/middlewares/auth.middleware.js (Lines 5-16)

- | Extracts cookie: sessionId = req.cookies.session
- | Queries Redis: session = await redis.get("session-<sessionId>")
- | Parses and attaches: req.user = JSON.parse(session)
- | req.user is now: { userid: "6832f5a1...", name: "Dhruv", email: "...", avatar: "..."
- | Calls next() → control passes to proxyWithHeader



[Step 5] gateway/utils/proxywithheader.js (Lines 9-11)

- | srcReq is the same req object from Step 4.
- | srcReq.user exists (set by protect middleware).
- | Reads srcReq.user.userid → "6832f5a1..."
- | Sets: proxyReqOpts.headers['x-user-id'] = "6832f5a1..."



[Step 6a] services/CHAT/controller/chat.controller.js (Lines 7, 18)

- | const userId = req.headers["x-user-id"] ← Reads the injected header
- | Uses userId to create conversations and query user-specific data.



[Step 6b] services/agent/controller/agent.controller.js

The agent service also receives this header (available in req.headers).
Currently it reads prompt and conversationId from req.body instead.

5. Where `proxyWithHeader` is Used (Integration Points)

A. Registration in Gateway (`gateway/index.js`)

```
// Line 9: Imported
import { proxyWithHeader } from "../utils/proxywithheader.js"

// Line 23: Applied to /chat route (after protect middleware)
app.use("/chat", protect, proxyWithHeader(process.env.CHAT_SERVICE))

// Line 24: Applied to /agent route (after protect middleware)
app.use("/agent", protect, proxyWithHeader(process.env.AGENT_SERVICE))
```

Key observation: `proxyWithHeader` is always used *after* `protect`. This is critical because `proxyWithHeader` depends on `req.user` which is set by `protect`. If `protect` rejects the request (401), `proxyWithHeader` never executes.

B. Contrast with the Auth Route

```
// Line 22: Auth route uses plain proxy – NO protect, NO header injection
app.use("/auth", proxy(process.env.AUTH_SERVICE))
```

The `/auth` route uses the plain `proxy()` directly because:

- Login/logout endpoints don't need authentication (the user isn't logged in yet during login).
- There is no `req.user` to inject — the session hasn't been created yet.

C. Downstream Consumers

Service	File	How it reads the header	What it does with it
Chat Service	<code>CHAT/controller/chat.controller.js</code> Line 7	<code>req.headers["x-user-id"]</code>	Creates conversation tied to this <code>userId</code>
Chat Service	<code>CHAT/controller/chat.controller.js</code> Line 18	<code>req.headers["x-user-id"]</code>	Queries conversation filtered by this <code>userId</code>
Agent Service	<code>agent/controller/agent.controller.js</code>	Header is available but not explicitly read	Agent uses <code>req.body</code> prompt/conversation

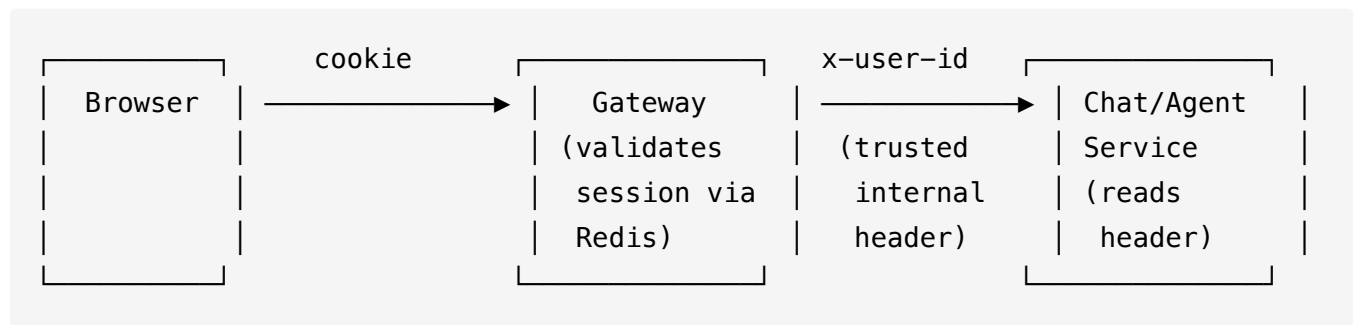
6. Why Does This Design Exist? (Architecture Rationale)

Problem it solves: Service Isolation

In a microservices architecture, each service runs independently. The Chat Service and Agent Service:

- Do **not** have access to Redis (only Gateway and Auth do).
- Cannot validate session cookies themselves.
- Have no way to know *who* is making a request.

The Solution Pattern: "Gateway Identity Injection"



- The **browser** sends a cookie (it cannot set custom headers like `x-user-id`).
- The **Gateway** translates the cookie into a trusted internal header.
- **Downstream services** trust the `x-user-id` header because it can only come from the Gateway (internal network, not exposed to the internet).

Why a header and not a query parameter or body field?

- **Headers are invisible** to the proxied request body — they don't interfere with the payload.
- **Headers are standard** for passing metadata between services in microservices patterns.
- The `x-` prefix is a convention for custom, non-standard headers.

7. File-to-File Execution Workflow

This is the exact sequence when a user clicks "New Chat" in the frontend, which triggers `GET /chat/create-conversation`:


```

[Browser] Sends GET /chat/create-conversation
  | Cookie header includes: session=<uuid>
  ▼
[gateway/index.js] Line 23: app.use("/chat", protect, proxyWithHeader(...))
  | URL matches "/chat" prefix.
  | Runs protect middleware FIRST.
  ▼
[gateway/middlewares/auth.middleware.js] protect()
  | Extracts sessionId from cookie.
  | Redis lookup: GET session-<uuid>
  | Parses JSON → attaches to req.user = { userid, name, email, avatar }
  | Calls next()
  ▼
[gateway/utils/proxywithheader.js] proxyReqOptDecorator callback fires
  | srcReq.user exists? ✅ YES (set by protect)
  | Reads srcReq.user.userid → "6832f5a1..."
  | Adds header: x-user-id = "6832f5a1..."
  | Returns modified proxyReqOpts
  ▼
[express-http-proxy] Forwards the request
  | Target: http://localhost:8002/create-conversation
  | Headers now include: x-user-id: "6832f5a1..."
  ▼
[services/CHAT/routes/chat.routes.js] Line 12: router.get("/create-conversation", createC
  | Route matched.
  ▼
[services/CHAT/controller/chat.controller.js] createConversation()
  | const userId = req.headers["x-user-id"] → "6832f5a1..."
  | await Conversation.create({ userId })
  | Returns new conversation document to Gateway → to Browser.

```

8. What Happens Without `proxyWithHeader` ?

If we replaced `proxyWithHeader(process.env.CHAT_SERVICE)` with the plain `proxy(process.env.CHAT_SERVICE)` :

What breaks	Why
<code>createConversation</code>	<code>req.headers["x-user-id"]</code> would be <code>undefined</code> → conversation created with <code>userId: undefined</code>
<code>getConversations</code>	<code>Conversation.find({ userId: undefined })</code> → returns empty array, user sees no conversations
All user-scoped data	Every conversation and message would be unlinked from any user — data isolation completely breaks

The app would appear to work (no crashes), but **no data would be tied to any user**. Every user would see nothing, or worse, see everyone's data mixed together.